

EduProva — Database Architecture KT

RDS PostgreSQL vs MongoDB — From Scratch Recommendation

Goal: Keep EduProva's architecture simple while reducing infrastructure and operational cost.

1. EduProva Core Data

Data / Service	Recommended	Why
Users	RDS PostgreSQL	Structured data and relationships.
Courses	RDS PostgreSQL	Connected to instructors, enrollments and other entities.
Enrollments	RDS PostgreSQL	Relational and transactional.
Orders	RDS PostgreSQL	Business transactions and relationships.
Payments	RDS PostgreSQL	Strong consistency and transactions.
Subscriptions	RDS PostgreSQL	Renewal/status/payment relationships.
Invoices / Contracts	RDS PostgreSQL	Structured financial/business records.
Flexible content	PostgreSQL JSONB	Flexible fields without another database initially.

2. Why Start With One Database?

- Two databases mean two infrastructure costs.
- Monitoring, backups, security, networking and maintenance become more complex.
- MongoDB is not currently integrated, so adding it only for the sake of using MongoDB does not reduce cost.
- Start with the minimum architecture that satisfies the requirements.

Key principle: Cost reduction means avoiding unnecessary infrastructure, not simply choosing the cheapest database product.

3. Can MongoDB Handle All EduProva Data?

Yes. MongoDB can technically handle Users, Courses, Orders, Payments, Subscriptions, Chat, Messages, Content and other workloads. MongoDB also supports ACID transactions, including multi-document transactions.

The decision is therefore about **data model and workload fit**, not whether MongoDB is capable of handling the data.

Workload	MongoDB fit	Main consideration
Users	Possible	Relationships need careful data-model design.
Payments	Possible with care	Consistency and transaction design are important.
Subscriptions	Possible with care	Status, renewal and payment relationships.
Enrollments	Possible with care	User/course relationships are relational in nature.
Chat / Messages	Strong fit	Document-oriented and potentially high-volume.
Activity	Strong fit	Large volumes of event/activity data.
Dynamic Content	Strong fit	Flexible document structure.

4. Does MongoDB Automatically Cause Latency?

No. MongoDB itself does not automatically make an application slow. Latency depends on network design, indexes, queries, connection pooling, cluster design and workload.

For MongoDB Atlas, private connectivity should be designed properly when low-latency private access is required.

5. Payment → Subscription → Enrollment

MongoDB can use a transaction for related updates:

Payment SUCCESS → Subscription ACTIVE → Enrollment ACTIVE → COMMIT

If these operations are spread across different databases, consistency becomes more complicated. Keeping tightly related financial/business data in one PostgreSQL database is simpler.

6. RDS PostgreSQL vs Aurora PostgreSQL

Both provide PostgreSQL functionality, so normal SQL, CRUD operations, joins, transactions, foreign keys and JSONB are available. The main differences are the underlying architecture, storage, scaling and availability capabilities.

Feature	RDS PostgreSQL	Aurora PostgreSQL
SQL / CRUD	Yes	Yes
Transactions	Yes	Yes
Foreign keys / joins	Yes	Yes
JSONB	Yes	Yes
Automated backups	Yes	Yes
Read replicas	Yes	Yes
HA options	Yes	Yes
Aurora-specific storage architecture	No	Yes
Cost-conscious starting point	Good fit	May cost more depending on configuration/workload

7. From-Scratch Recommendation

For EduProva, start with RDS PostgreSQL.

Initial architecture:

EKS → RDS PostgreSQL

Use RDS PostgreSQL for:

- Users
- Courses
- Enrollments
- Orders
- Payments
- Subscriptions
- Invoices
- Contracts

Use **PostgreSQL JSONB** for flexible content/metadata where appropriate.

8. When to Add MongoDB

- Chat/messages become very high-volume.
- Activity/event data grows significantly.
- Dynamic document data becomes difficult to manage efficiently with PostgreSQL JSONB.
- Load testing or real production metrics show a clear MongoDB benefit.

MongoDB should be requirement-driven, not technology-driven.

9. Cost-Saving Strategy

Phase	Architecture	Purpose
1	EKS → RDS PostgreSQL	Minimum infrastructure and operational complexity.
2	RDS PostgreSQL + JSONB	Handle flexible data without another database.
3	Add MongoDB only if justified	Separate only workloads that gain a clear technical benefit.
4	Measure and optimize	Monitor CPU, connections, IOPS, storage, latency and query performance.

10. Future Migration Path

Starting with RDS PostgreSQL does not prevent moving to Aurora later if scale, HA or architecture requirements justify it.

Today: EKS → RDS PostgreSQL → EduProva core data

Future: EKS → Aurora PostgreSQL → EduProva core data

11. KT / Interview Answer

“For EduProva, I would start from scratch with RDS PostgreSQL because our core data such as users, courses, enrollments, orders, payments and subscriptions is relational and transactional. To reduce cost and operational complexity, I would avoid deploying MongoDB from day one. PostgreSQL JSONB can handle flexible content initially. If the application later develops a clear requirement for high-volume chat, activity streams or highly dynamic document data, we can introduce MongoDB selectively.”

12. Final Decision

Area	Decision
Primary database	RDS PostgreSQL
Flexible data	PostgreSQL JSONB initially
MongoDB	Add later only if workload justifies it
Aurora	Consider later when scale/HA requirements justify it
Cost strategy	Start simple → measure → scale → add technology only when needed

Final takeaway

One database that satisfies the current requirements is simpler and usually more cost-conscious than deploying two databases before there is a real need.

Prepared from the provided EduProva database architecture discussion.